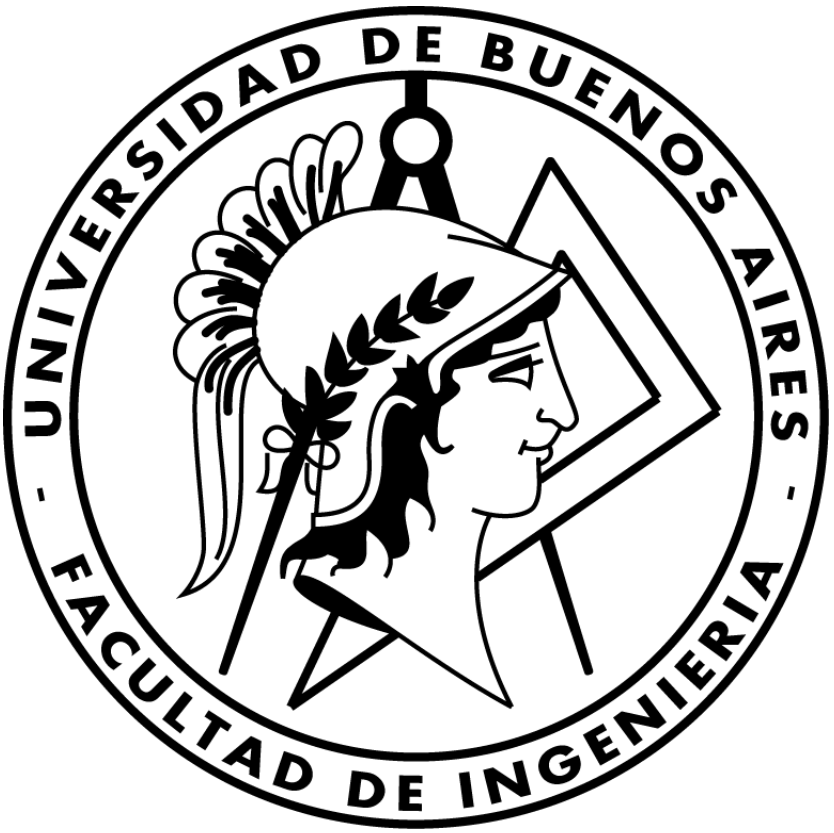


Trabajo Práctico Grupal - Need for Speed

Taller de Programación (TA045)

Curso 1 - Veiga

2C 2025

Grupo 1

Nombre	Apellido	Padrón
Mateo	Zorzi	109634
Nestor	Palavecino	108244
Cristian	Ledesma	111426
Brian	Céspedes	108219

Documentación Técnica del Proyecto

En este documento se describe la arquitectura, diseño e implementación del videojuego Need for Speed para la materia Taller de Programación de la Facultad de Ingeniería - Universidad de Buenos Aires.
El juego consiste en carreras dentro de un mapa con vista estilo *top-down*, donde cada jugador selecciona un automóvil, se une o crea una partida, elige carreras y participa en un circuito administrado por un servidor.

El sistema utiliza una arquitectura cliente-servidor, donde:

El servidor administra

- Conexiones de múltiples clientes.
- Múltiples partidas en simultáneo.
- Selección de autos/carreras.
- Generación de un mundo con colisiones y demás objetos útiles para el correcto renderizado del lado del cliente.
- Simulación de posiciones en dicho mundo.
- Ciclo de juego (game loop).
- Envío de snapshots del estado del mundo.

El servidor posee 4 hilos

- Acceptor -> Acepta nuevas conexiones.
- ClientHandler -> Maneja peticiones del cliente al estar en el menú principal, también se encarga de disparar el Sender y el Receiver (otro hilo) cuando empieza el juego.
- GameLoop -> (no se bloquea) maneja la lógica general del juego, envío de snapshots, actualización del estado del mundo.
- Receiver (1 por cliente) -> Agrega (push) comandos en la queue global de la partida.
- Sender (no es un hilo) -> Extrae (pop) las snapshots en cada instante de la partida.

El cliente administra

- Interfaz gráfica (Qt para menús, y SDL para renderizar el juego).
- Entrada del usuario (teclado y mouse).
- Reproducción de sonidos.
- Renderizado de autos, mapa, minimapa, HUD.
- Comunicación con el servidor mediante un protocolo binario

El cliente posee 3 hilos

- UILoop -> (no se bloquea, detecta la entrada del usuario y renderiza)
- Sender -> envía las interacciones al servidor (queue -> socket).
- Receiver -> recibe las snapshots del servidor (socket -> queue).

Inicialmente, todos los clientes entran a un Main Menu o Lobby, el cual se utiliza para

- Introducir un nombre de usuario.
- Crear, unirse o listar partidas.
- Seleccionar distintas carreras al crear una partida.

Para manejar actualizaciones del lobby se utiliza una clase

LobbyReceiver (cliente)

Es un hilo que:

- Escucha acciones del servidor y emite señales Qt para cerrar o avanzar pantallas.
- Evita bloquear el UI Loop mientras se esperan mensajes del servidor.

Se utiliza un protocolo binario claramente establecido

- Primer byte -> acción a realizar.
- Datos específicos de cada acción.
- Serialización binaria (se utilizan tipos de datos uint8, uint16, uint32, string length (uint32) + bytes, etc.).

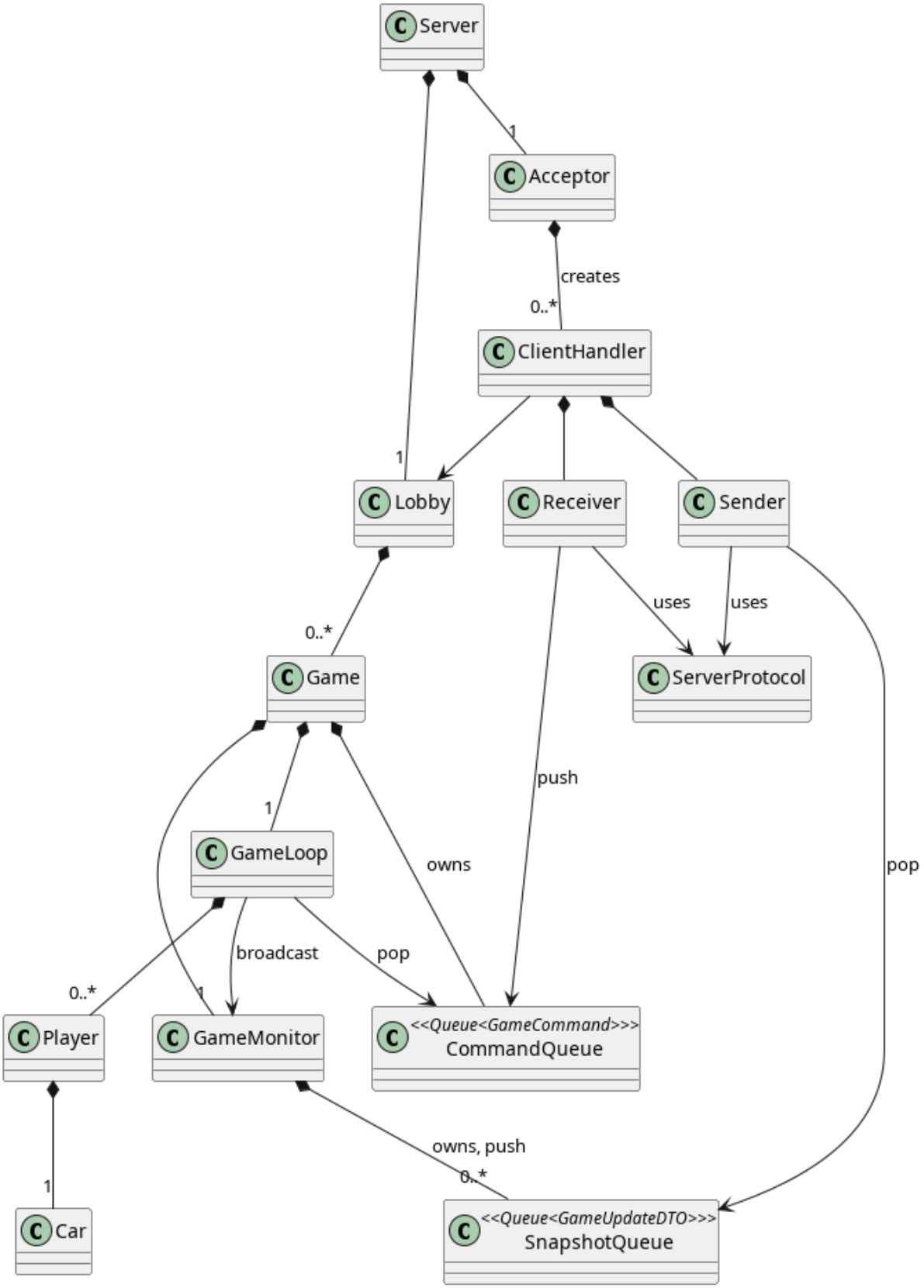
Ejemplos de mensajes enviados por el cliente

- SEND_USERNAME.
- SEND_CAR_MOVEMENT.
- CREATE_GAME.
- JOIN_GAME.
- GET_CAR_CATALOGUE.
- SEND_CAR_SELECTION.
- SEND_RACE_SELECTION.

Ejemplos de datos enviados por el servidor

- ID del cliente.
- Catálogo de autos.
- Catálogo de carreras.
- Listado de partidas,
- Snapshot del mundo (WorldSnapshotDTO).
- Posiciones individuales (CarPositionDTO).
- Checkpoint siguiente.
- Leaderboard.

Diagrama UML servidor



Motor físico

El juego utiliza como motor físico a Box2D. Esto abstrae enormemente al desarrollador de realizar cálculos complejos o vectoriales para simular la física del mundo virtual, lo cual puede ser una tarea bastante tediosa.

Box2D en este proyecto se utiliza para:

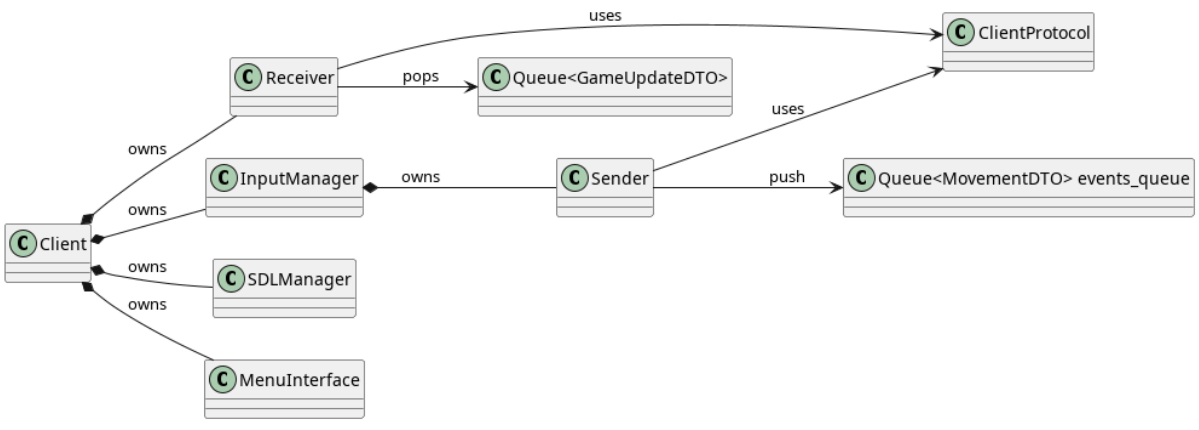
- Detección de colisiones.
- Parámetros físicos del auto (aceleración, velocidad máxima, masa, etc.).
- Sensores para detectar cuando un jugador entra/sale de cierta zona.

Renderizado y SDL (cliente)

El cliente utiliza **SDL2** para renderizar el juego durante las carreras que se desarrollan. Para facilitar su uso, se emplea el wrapper para C++ **SDL2pp** que ofrece RAII y manejo seguro de recursos. El renderizado es top-down 2D y el modulo grafico se encarga de:

- Crear y administrar la ventana del juego y el renderer principal.
- Cargar recursos como texturas.
- Renderizar los autos según su posición y rotación recibida en cada actualización del servidor.
- Dibujar el mapa según el circuito seleccionado, compuesto por:
    - Imagen base del mapa.
    - Los recortes de decoraciones y puentes por los que el jugador puede pasar por debajo.
- Renderizar elementos adicionales:
    - Minimapa y posición relativa del jugador.
    - Próximo checkpoint visible.
    - Leaderboard y tienda de mejoras (eventuales).
    - HUD.

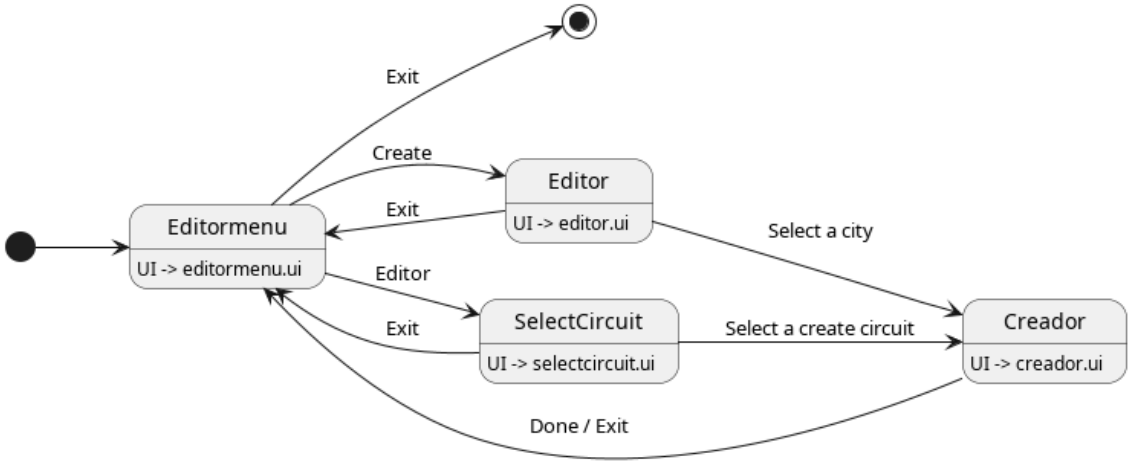
Diagrama UML cliente



Editor

El editor cuenta con dos funciones principales, el Creador de nuevos circuitos (Editormenu -> Editor -> Creador) y el editor de circuitos ya creados (Editormenu -> SelectCircuit->Creador).

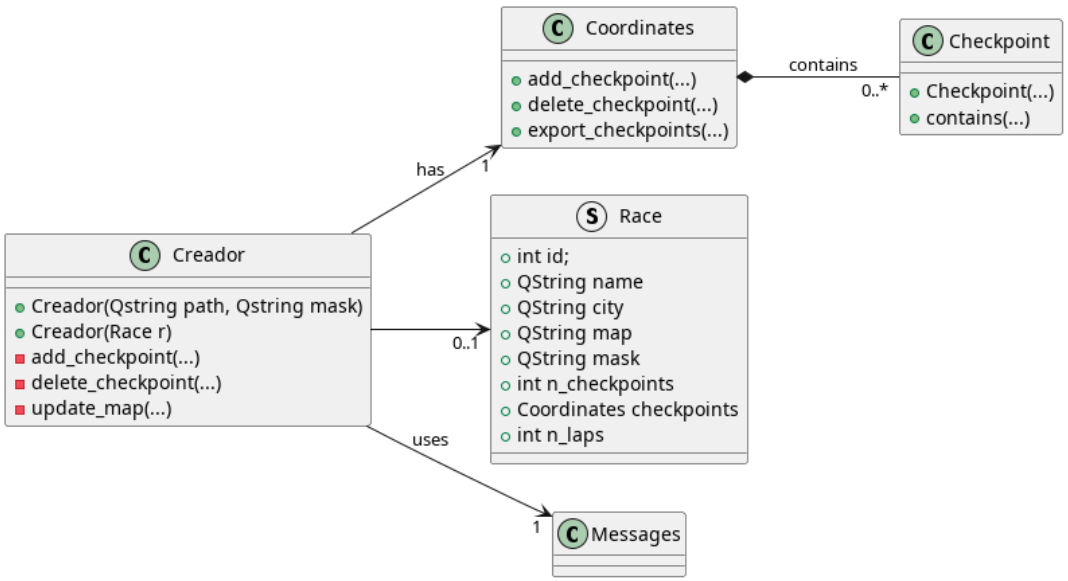
En este diagrama de estados se gráfico el flujo entre las ventanas del editor. Se puede observar como sería el flujo de creación o edición de un circuito, pasando por las pantallas correspondientes al menú Editor.



Descripción de cada pantalla:

- **Editormenu:** Presenta el menú principal del editor, su principal función es redirigir al usuario al creador o editor de circuitos.
- **Editor:** Primera pantalla del creador de nuevos circuitos, se le pide al usuario elegir en qué ciudad se creará el circuito.
- **SelectCircuit:** Primera pantalla del editor de circuitos previamente creados, se le lista al usuario las carreras creadas y guardadas en el servidor.
- **Creador:** es el núcleo funcional del editor, donde se implementa toda la lógica de edición de circuitos. Desde este módulo se puede:
    - Generar un **mapa vacío** cuando se está creando un circuito nuevo.
    - **Cargar** un circuito ya existente para su modificación.
    - Editar propiedades esenciales del circuito, como:
        - Nombre del circuito
        - Cantidad de vueltas
        - Ubicación y cantidad de checkpoints

    Una vez concluida la edición, el usuario puede **exportar el archivo del circuito**, el cual es utilizado posteriormente por el juego principal.



Exportación archivo del circuito

Formato del archivo creado al exportar un circuito. En un archivo csv con el siguiente formato:

"nombre_carrera".csv

"

"path_mapa_ciudad", "path_mascara_ciudad"

city,n_laps (header)

"nombre de la ciudad", "número de vueltas"

id, orientacion, xCenter, yCenter, height, width(header)

"información checkpoints"

"

Uso de mutex y acceso concurrente

Al hacer uso de múltiples hilos, necesitamos garantizar que no existan race conditions. Por ello, se utilizan mutex en varios puntos del código, pero únicamente para proteger recursos compartidos.

La clase Lobby (server-side) por ejemplo, administra un mapa <game_id, Game>. Dos o más jugadores podrían intentar ingresar a una misma partida al mismo tiempo, lo cual supondría un problema de concurrencia. Existe un único Lobby en todo el server para todas las partidas.

La clase GameMonitor (server-side) administra un mapa <game_id, Queue (snapshot_queue)>. Si bien cada cliente tiene su propia queue de snapshots, las mismas son administradas por este objeto. Se encarga de "registrar" una nueva queue en su mapa, "desregistrar" la misma, hacer el broadcast. Existe un GameLoop y un GameMonitor por partida.

Sockets

Los sockets son sumamente importantes en este proyecto, ya que permiten la comunicación entre cliente y servidor. Para ello, se utiliza una clase dedicada (créditos a Martin Di Paola por brindarnos su implementación). Todas las operaciones de sockets son bloqueantes, por lo que no se deben utilizar en los hilos GameLoop ni UILoop.

Créditos

Martín Di Paola, por ser quién implementó los Sockets que se utilizan en este proyecto. A su vez, se usó su implementación de Threads y Queues Thread Safe.